

Matthew Duffy
Prof. Arun Sharma
Applied AI and Machine Learning
December 10th, 2025

Advanced Topics – Final Project Part 5

Topic 1 – Access-control and security (Creation of roles in SQL can be found in roles.sql)

I created two different roles (can be found in roles.sql), *commish* and *manager* and have them different grants. The *commish* or commissioner have read and write access to all the schema since they oversee the league. Managers have read access to manager_info and player_info schemas, but not transactions; they have no write access. In theory, the managers should have less oversight into what other managers are doing transaction wise than the commissioner.

Examples

```
Query | Query History
1 Set role manager
2 ;
3
4 Select * From transactions.trades
5 ;

Data Output | Messages | Notifications

ERROR: permission denied for schema transactions
LINE 1: Select * From transactions.trades
              ^

SQL state: 42501
Character: 15
```

Query	Query History
1	<code>Set role commish</code>
2	<code>;</code>
3	
4	<code>Select * From transactions.trades</code>
5	<code>;</code>

Data Output	Messages	Notifications
SQL		
Showing rows: 1 to 258		Page No: 1

	season integer	league_id [PK] character varying (50)	trade_id [PK] integer	timestamp timestamp with time zone	status character varying (50)	from_team character varying (100)	from_team_key character varying (100)	to_team character varying (100)	to_team_key character varying (100)
1	2023	423.1.927065	273	2023-11-16 12:10:31-05	successful	Griddy Goblin	423.1.927065.1.2	I HAVE 0 HOPE	423.1.927065.1.1
2	2023	423.1.927065	273	2023-11-16 12:10:31-05	successful	Griddy Goblin	423.1.927065.1.2	I HAVE 0 HOPE	423.1.927065.1.1
3	2023	423.1.927065	273	2023-11-16 12:10:31-05	successful	I HAVE 0 HOPE	423.1.927065.1.1	Griddy Goblin	423.1.927065.1.2
4	2023	423.1.927065	265	2023-11-15 08:38:03-05	successful	Bust on Bijan	423.1.927065.1.10	John's Wondrous Team	423.1.927065.1.4
5	2023	423.1.927065	265	2023-11-15 08:38:03-05	successful	Bust on Bijan	423.1.927065.1.10	John's Wondrous Team	423.1.927065.1.4
6	2023	423.1.927065	265	2023-11-15 08:38:03-05	successful	John's Wondrous Team	423.1.927065.1.6	Bust on Bijan	423.1.927065.1.10
7	2023	423.1.927065	248	2023-11-12 12:39:28-05	successful	David's Definitive Team	423.1.927065.1.6	John's Wondrous Team	423.1.927065.1.4

Query
Query History

```

1 Set role manager
2 ;
3
4 INSERT INTO manager_info.managers (manager_guid, manager_nickname)
5 VALUES
6 ('RLDAQUDDIH63JIUDYHUH7XHVVE', 'Miguel'),
7 ('BB5X53X74T&BUFHGW4ZE3GFS2I', 'Jimmy'),
8 ('HL20CHLHFHJFYUVVENZETP5I', 'Sam');
9 ;|

```

Data Output
Messages
Notifications

ERROR: permission denied for table managers

SQL state: 42501

Query
Query History

```

1 Set role manager
2 ;
3
4 Update player_info.detailed_roster
5 Set position_type = '0'
6 Where Season = 2023
7 and Week = 1
8 and player_id = 27540
9 ;

```

Data Output
Messages
Notifications

ERROR: permission denied for table detailed_roster

SQL state: 42501

Query
Query History

```

1 Set role commish
2 ;
3
4 Update player_info.detailed_roster
5 Set position_type = '0'
6 Where Season = 2023
7 and Week = 1
8 and player_id = 27540
9 ;

```

Data Output
Messages
Notifications

UPDATE 1

Query returned successfully in 55 msec.

Query
Query History

```

1 Set role manager
2 ;
3
4 Select * from player_info.detailed_roster
5 Where Season = 2023
6 and Week = 1
7 and player_id = 27540
8 ;|

```

Data Output
Messages
Notifications

Showing rows: 1 to 1
Page No

	season [PK] integer	week [PK] integer	team_key character varying (100)	team_name character varying (100)	manager_guid [PK] character varying (50)	manager_name character varying (50)	player_id [PK] integer	player_name character varying (100)
1	2023	1	423.I927065.t.8	nooch	2LZKPL56AY3GJ5QCXA6HWZOMGA	nick	27540	Odell Beckham

These screenshots show that manager cannot read transactions.trades and cannot write to manager_info.managers nor player_info.detailed_roster, while commish can write to player_info.detailed_roster and read transactions.trades.

HIPPA/GLBA Discussion

HIPPA (Health Insurance Portability and Accountability Act) does not apply to this fantasy football dataset. HIPPA applies to Protected Health Information (PHI) maintained by covered entities like hospitals, clinics, insurance companies, and etc.. Fantasy football data does not consist of that whatsoever; the database has information on my fantasy football league (manager_info and transactions) and NFL player data (player_info). Therefore, HIPPA does not apply to this database at all.

GLBA (Gramm–Leach–Bliley Act) also does not apply to this fantasy football dataset. GLBA is a federal law that protects people's nonpublic personal information (NPI) handled by financial institutions. It regulated how financial institutions collect, use, and store this sensitive data. Under GLBA, financial institutions must have security plans for sensitive data, provide notice of their privacy policies to customers, and allow customers to opt out of data sharing with third parties. The protected data consists of credit card numbers, social security numbers, income balances, and much more financial information. This regulation should help mitigate crimes like identity theft and ensures consumer privacy. I am not a financial institution and do not have any financial data in the database. Yahoo fantasy is free to use, so no credit cards are stored in the application at all.

While these specific regulations do not apply to the fantasy football database, there are important concepts that should still apply to the database like consent and general security. The NFL information is fully public, so there is not much to do their security wise, but the yahoo specific data is about my friends and me. When my friends decided to join the league, they were not aware that someone would be tracking their activity in the league. I let everyone in my league know that I started doing this and asked if anyone had an issue with it, so I do believe I got consent for the data. Moreover, I maintain this database locally now, so there is no way for me to share this with other people unless I move it to an online location, but if I do give access to my friends so they can see the data for themselves, the role permissions will be essential. My friends are not data people, so I would be weary to give them any write access to the database. They could easily start trying to do something and start dropping tables out of nowhere, which is why they need a specific role that limits them to just inspecting the data. In that way, the roles will ensure the integrity of the database! One other thing I could do is encrypt names if my friends wanted a bit more privacy. The databases main use case is reporting interesting statistics about our league, and there is no sensitive data in the database, so it definitely not necessary at this point. But maybe one day I release this to show other people how to track this sort of data; that would change things and maybe cause the league members (including me) to want their names anonymized in the database. However this project progresses, it will be important for me to keep my friends updated on what I am doing so they are aware of how their data is being used and can alter their participation as they see fit.

Topic 2 – NoSQL (Code can be found in MercyFantasyDatabase.ipynb – NoSQL Replication)

I took data from the PostgreSQL database and uploaded it into MongoDB in python (using pymongo). I was simply able to query the entire table for managers, manager_seasons, matchups, and detailed_roster and insert them into NoSQL equivalents. I also wrote a query to convert SQL data into a JSON string commonly found in NoSQL as a document and uploaded it into a collection called manager_rosters. From there, I wrote a few different queries in NoSQL and SQL and compared performance results.

The first query I ran was a simple search on detailed roster to find a particular player, week, and season. There is an index on season, week, and player_id, and a corresponding one in PostgreSQL called detailed_roster_pkey since all primary keys have an index (which led to both queries only evaluating the one record they returned). This simple task led to extremely low execution times, nearly 0 ms in MongoDB and 0.232 ms in PostgreSQL. The very small difference may be due to MongoDB materializing a BSON object to run the query or simple issues measuring sub second times. All in all, these queries are both highly efficient for this very simple task! The optimization time for PostgreSQL was 3.387 ms and around 3 ms for MongoDB, so very similar amounts of time to optimize the query before running. At the very least, this example shows how highly optimized both databases are. With the same indexes available, both databases are taking the same steps to retrieve the record as fast as possible.

The next query performed an aggregated join, which is more computationally complex than the prior example. PostgreSQL uses nested loops and a technique called memoization (optimization technique which stores the results of expensive function calls and returns the cached result when the same inputs occur again) to avoid repeated index scans, which saves time when joining. MongoDB's total execution time of 439 ms is significantly higher than PostgreSQL's 54.639 ms, but the difference is not caused by MongoDB examining more rows. Both systems examine roughly 75,000 candidate rows, but PostgreSQL applies a more efficient execution strategy via its use of memoization and an index whose leading columns align with the join conditions. Both systems have low planning times as well (3.187 ms for PostgreSQL and about 0 for MongoDB). PostgreSQL performs an index range scan on (season, week) and then filters on team_key in memory, which reduces each lookup to approximately 17 matching rows after filtering, even though about 167 rows are inspected per lookup. MongoDB on the other hand performs a lookup for each outer document and must scan many more candidate documents in the joined collection due to the missing index, resulting in substantially more work in query. Both queries aggregate to the season-team level and return 1,380 rows. It seems clear that join heavy queries are better in traditional SQL compared to NoSQL, due to memoization and the ability to use indexes effectively.

For the final query I pulled all player a particular manager (me) has had on their roster. In PostgreSQL I queried the detailed_roster table which has one player per week. In MongoDB, I simply queried the document from the JSON constructed collection manager_rosters, which contains one document per manager with the complete historical roster embedded directly inside the document. Both queries are able to leverage indexes on the search field for optimized execution, but MongoDB only needed to locate a single indexed document, so returned it in nearly 0 ms (optimization planning also took about 0 ms). That illustrates the strengths of the NoSQL document model: storing related, semi-structured data in a single document using key-value structures can make certain retrieval patterns extremely fast, especially in bid data and real time analytics where large amounts of related information can be with one key lookup. Postgres

took around 2.1 ms (optimization planning took about 2.3 ms); it utilized a bitmap index scan on `manager_guid` and examined/returned 874 rows in the process. While this is highly efficient, PostgreSQL still needs to retrieve every entry while MongoDB only needs to find and return one document. These time gap would be expounded as well as table sizes grow in PostgreSQL!

NoSQL has many cases where it simply outshines traditional SQL databases. NoSQL excels in high read environments and where data is often unstructured. Fetching large amounts of data is also easier when it is all sitting in one place waiting to be retrieved as a unit. Flexible schemas allow for databases to easily grow horizontally and adapt to new challenges easily. But traditional SQL databases are still superior when it comes to databases that often utilize joins. NoSQL documents can remove the need for joins but depending on the nature of the business/field, joins may become necessary as different teams need the data. ACID requirements are often limited in NoSQL but guaranteed on traditional SQL systems. These systems shine in different areas, so for data scientists/engineers it is mostly important to know what the differences are so that we can chose the right one for the task at hand.